

Département Informatique et Télécommunications

Chaîne de Vérification de Modèles de Processus



Réalisé par :

Cheikh Brahim Jedou
Oussama El Boubekri

Encadrant : M. Thierry Loic

Table des matières

1	Introduction générale	3
1.1	Contexte et objectif du mini-projet	3
1.2	Organisation de l'équipe et répartition des tâches	3
1.3	Outils et technologies utilisés	3
1.4	Chaîne de vérification globale : $PDL3 \rightarrow SimplePDL \rightarrow PetriNet \rightarrow TINA$ (LTL)	4
2	Modélisation des procédés	5
2.1	Introduction	5
2.2	Le métamodèle SimplePDL	5
2.3	Le métamodèle PetriNet	6
2.4	Lien entre les deux métamodèles	7
3	Sémantique statique des métamodèles	8
3.1	Principe et mise en œuvre	8
3.2	Règles codées pour <i>SimplePDL</i> (dans <i>SimplePDLValidator</i>)	8
3.3	Règles codées pour <i>PetriNet</i> (dans <i>PetriNetValidator</i>)	8
3.4	Extrait de code (vérification sur Arc)	9
4	Transformation Modèle à Modèle	10
4.1	Objectif et principe des transformations	10
4.2	Transformation $PDL3 \rightarrow SimplePDL$ (ATL)	10
4.3	Transformation $SimplePDL \rightarrow PetriNet$ (ATL)	11
5	Transformation Modèle à Texte (Acceleo)	13
5.1	Principe général	13
5.2	Template <code>toHTML.mtl</code> : génération d'une vue HTML	13
5.3	Template <code>toDOT.mtl</code> : export Graphviz DOT	13
5.4	Template <code>toTina.mtl</code> : génération du fichier <code>.net</code>	13
6	Éditeurs et visualisations	17
6.1	Rappels technologiques	17
6.2	Éditeur graphique Sirius	17
6.3	Éditeur textuel Xtext	18
6.4	Intégration Eclipse : édition et vérification	19
7	Vérification formelle avec TINA / SELT	20
7.1	Introduction à la logique temporelle linéaire (LTL)	20
7.2	Compilation et génération du fichier <code>.ktz</code>	20

7.3	Vérification des propriétés LTL	20
8	Partie faite après la soutenance orale	21
	Conclusion générale	22

Chapitre 1

Introduction générale

1.1 Contexte et objectif du mini-projet

Ce mini-projet s'inscrit dans le cadre du module **Ingénierie Dirigée par les Modèles (IDM)**. L'objectif principal est de concevoir une **chaîne complète de modélisation, transformation et vérification** d'un **processus** à travers différents niveaux d'abstraction, allant de la description textuelle d'un procédé jusqu'à sa vérification formelle par des outils dédiés.

Le projet repose sur l'utilisation successive de plusieurs technologies de la plateforme **Eclipse Modeling** afin de définir un langage de modélisation de processus, de le transformer en un modèle exécutable, puis d'en vérifier le comportement à l'aide d'un réseau de Pétri et d'outils d'analyse tels que **TINA** et **SELT**.

Ainsi, notre objectif était de développer une chaîne allant de la spécification à la vérification, en passant par la transformation de modèles, selon le schéma suivant :

$$PDL3 \longrightarrow SimplePDL \longrightarrow PetriNet \longrightarrow TINA(LTL).$$

1.2 Organisation de l'équipe et répartition des tâches

Le travail a été réalisé en **binôme**, dans un esprit de collaboration et de complémentarité. Chaque membre du groupe a contribué activement à toutes les étapes du projet, que ce soit lors de la conception des métamodèles, de l'écriture des transformations ATL, ou encore de la génération du code avec Acceleo et de la vérification sous TINA.

Nous avons choisi de travailler de manière **collaborative et interactive** : chaque tâche était d'abord discutée et planifiée ensemble, puis répartie selon les compétences et préférences de chacun. Lorsque l'un rencontrait une difficulté, l'autre l'aidait à la surmonter, permettant ainsi une progression continue et équilibrée.

Ce mode de travail nous a permis de renforcer notre compréhension mutuelle des concepts IDM tout en garantissant la cohérence globale du projet.

1.3 Outils et technologies utilisés

Pour la mise en œuvre de ce projet, nous avons utilisé plusieurs outils et frameworks issus de l'écosystème Eclipse :

- **Ecore** / **EMF** : pour la définition des métamodèles (**SimplePDL** et **PetriNet**) ;

- **Xtext** : pour la création de la syntaxe textuelle (PDL3) ;
- **Sirius** : pour la création d'un éditeur graphique intuitif du modèle SimplePDL ;
- **ATL** : pour les transformations de modèles ($\text{PDL3} \rightarrow \text{SimplePDL}$ puis $\text{SimplePDL} \rightarrow \text{PetriNet}$) ;
- **Acceleo** : pour la génération automatique de code texte (fichiers `.net` compatibles TINA) ;
- **TINA** / **SELT** : pour la simulation et la vérification formelle des propriétés LTL ;
- **Java (EValidator)** : pour la vérification de la sémantique statique des métamodèles.

Ces outils ont été utilisés de manière complémentaire afin de couvrir l'ensemble du cycle de vie du modèle : depuis sa création jusqu'à sa vérification finale.

1.4 Chaîne de vérification globale : $\text{PDL3} \rightarrow \text{SimplePDL} \rightarrow \text{PetriNet} \rightarrow \text{TINA (LTL)}$

La chaîne de vérification développée repose sur quatre grandes étapes successives :

1. **Modélisation textuelle (PDL3)** : définition du processus au moyen d'une syntaxe textuelle conçue avec Xtext ;
2. **Transformation vers SimplePDL** : traduction du modèle PDL3 en un modèle SimplePDL plus structuré et conforme au métamodèle EMF ;
3. **Transformation vers PetriNet** : génération d'un réseau de Pétri à partir du modèle SimplePDL pour représenter le comportement dynamique du processus ;
4. **Génération et vérification** : conversion du modèle PetriNet en un fichier `.net` utilisable par TINA, puis vérification de propriétés temporelles (LTL) avec SELT.

Cette chaîne illustre concrètement le principe de l'**ingénierie dirigée par les modèles** : chaque niveau de modélisation apporte une abstraction différente, et les transformations assurent la cohérence entre ces niveaux jusqu'à l'analyse finale.

Chapitre 2

Modélisation des procédés

2.1 Introduction

La modélisation d'un procédé constitue une étape essentielle de notre projet. Elle permet de représenter de manière abstraite et cohérente le déroulement d'un processus, les activités qui le composent ainsi que les dépendances entre celles-ci. Dans notre mini-projet, nous nous sommes appuyés sur deux métamodèles principaux :

- le métamodèle **SimplePDL**, servant à représenter la structure logique et statique d'un processus ;
- le métamodèle **PetriNet**, qui modélise le comportement dynamique du processus sous forme de réseau de Pétri.

Ces métamodèles ont été définis à l'aide de la plateforme *Eclipse Modeling Framework (EMF)*, qui permet de créer des structures orientées objet pour la conception de langages spécifiques. Ils forment la base sur laquelle repose l'ensemble de la chaîne de transformation et de vérification développée par la suite.

2.2 Le métamodèle SimplePDL

Le métamodèle **SimplePDL** (*Simplified Process Description Language*) décrit la structure statique d'un procédé. Il a pour but de représenter les différentes activités (ou tâches), leurs relations de dépendance, ainsi que les ressources nécessaires à leur exécution.

Évolution du métamodèle

Au départ, le métamodèle SimplePDL ne comportait que les éléments de base : *Process*, *WorkDefinition* et *WorkSequence*. Cette version permettait de décrire des chaînes de tâches et leurs dépendances logiques, mais elle ne prenait pas en compte les contraintes matérielles ou humaines pouvant influencer l'exécution du processus.

Au fur et à mesure de l'avancement du projet, nous avons constaté que certaines tâches nécessitaient l'utilisation de ressources limitées (par exemple des machines, des outils ou du personnel). Pour refléter cette réalité, nous avons donc enrichi le métamodèle en ajoutant deux nouvelles classes :

- **Ressource**, décrivant les ressources disponibles et leur quantité totale ;
- **RessourceUsage**, permettant d'associer une tâche à une ressource et de spécifier la quantité utilisée.

Cette extension a rendu le métamodèle plus réaliste et plus conforme aux besoins d'un processus industriel ou organisationnel.

Structure du métamodèle

Le métamodèle complet est composé des classes suivantes :

- **Process** : classe racine représentant le procédé dans son ensemble. Elle contient un ensemble d'éléments appelés *ProcessElements*.
- **WorkDefinition** : représente une activité ou une tâche à exécuter dans le processus.
- **WorkSequence** : relie deux tâches selon un type de dépendance défini par l'énumération *WorkSequenceType*.
- **Ressource** : décrit les ressources disponibles, caractérisées par un nom et une quantité totale.
- **RessourceUsage** : relie une ressource à une activité en précisant la quantité consommée.

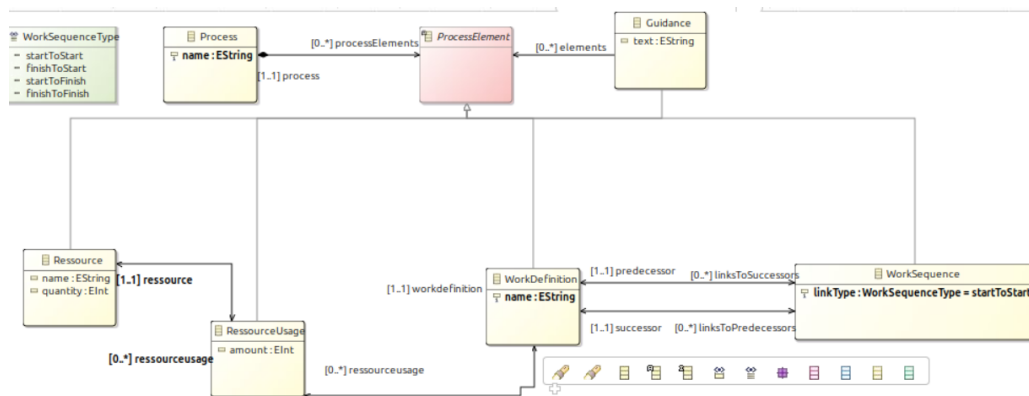


figure Métamodèle *SimplePDL* après ajout des ressources

Commentaires

L'ajout des ressources a permis de rendre le modèle plus expressif et de préparer la gestion des contraintes de capacité dans la phase de vérification. Ainsi, il est désormais possible de modéliser des situations où deux tâches concurrentes ne peuvent pas s'exécuter simultanément faute de ressources disponibles. Cette amélioration a été un point clé de la cohérence globale de notre chaîne de transformation.

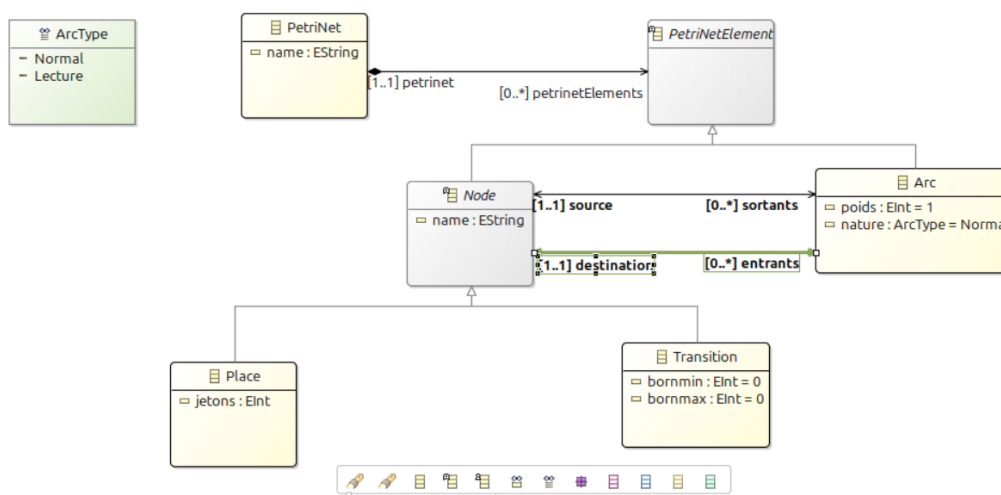
2.3 Le métamodèle PetriNet

Afin de pouvoir simuler et vérifier le comportement d'un processus, nous avons conçu un second métamodèle : **PetriNet**. Celui-ci repose sur les principes des réseaux de Pétri, très utilisés pour la modélisation de systèmes dynamiques concurrents.

Structure du métamodèle

Le métamodèle PetriNet comprend les éléments suivants :

- **PetriNet** : élément racine qui regroupe l'ensemble des composants du réseau ;
- **Place** : représente un état du système ou une condition. Chaque place contient un certain nombre de jetons (**jetons**) indiquant l'état courant ;
- **Transition** : correspond à une action ou un événement qui modifie le marquage du réseau ;
- **Arc** : relie une place et une transition. Les arcs peuvent être de nature **Normal** ou **Lecture** selon leur rôle, et portent un poids (**poids**) indiquant la quantité de jetons transférée.



figureMétamodèle *PetriNet*

Commentaires

Le métamodèle PetriNet offre une vision dynamique du processus. Chaque tâche du modèle SimplePDL correspond, après transformation, à un sous-réseau composé de places et de transitions : **ready**, **started**, **running** et **finished**, reliées par des transitions **start** et **finish**. Les ressources sont représentées par des places supplémentaires dont le nombre de jetons traduit la quantité disponible.

Ce modèle comportemental permet ensuite d'analyser la bonne exécution du processus à l'aide d'outils de vérification comme TINA.

2.4 Lien entre les deux métamodèles

Le métamodèle SimplePDL décrit les aspects statiques d'un processus, tandis que PetriNet en formalise les aspects dynamiques. La transformation du premier vers le second permet de passer d'une description conceptuelle à une représentation exécutable. Ainsi, chaque élément du modèle SimplePDL est traduit en un ensemble de places, transitions et arcs, formant un réseau de Pétri complet et cohérent. Cette correspondance constitue la base de la chaîne de transformation et de vérification développée dans la suite du projet.

Chapitre 3

Sémantique statique des métamodèles

3.1 Principe et mise en œuvre

La **sémantique statique** correspond à l'ensemble des règles de validité vérifiées avant toute transformation ou exécution. Dans notre projet, nous avons implémenté ces règles en **Java** au sein de deux validateurs : **SimplePDLValidator** (pour le métamodèle SimplePDL) et **PetriNetValidator** (pour le métamodèle PetriNet). L'exécution se fait directement dans Eclipse via l'action *Validate*, avec des messages d'erreur précis sur les éléments fautifs.

3.2 Règles codées pour *SimplePDL* (dans SimplePDLValidator)

- **Ressource** :
 1. le nom ne doit pas être vide ;
 2. la quantité doit être **strictement supérieure à 0**.
- **RessourceUsage** :
 1. amount doit être **strictement supérieur à 0** ;
 2. amount doit être **inférieur ou égal** à la quantité de la ressource associée.
- **WorkDefinition** :
 1. il ne peut pas exister **deux tâches portant le même nom** dans un même Process.

3.3 Règles codées pour *PetriNet* (dans PetriNetValidator)

- **Arc** :
 1. un arc ne peut pas relier **deux places** ni **deux transitions** ; il doit relier une **Place** et une **Transition** (dans un sens ou l'autre) ;
 2. le **poids** de l'arc doit être **greater or equal to 1** (≥ 1).
- **Place** :
 1. le nombre de **jetons** doit être **strictement supérieur à 0** (pas de place vide au marquage initial) ;
 2. le **nom** d'une place ne doit pas être vide ;

3. il ne peut pas exister **deux places du même nom** dans un même réseau de Pétri.

— **Transition :**

1. le **nom** d'une transition ne doit pas être vide ;
2. il ne peut pas exister **deux transitions du même nom** dans un même réseau de Pétri.

3.4 Extrait de code (vérification sur Arc)

Exemple de vérification Java dans PetriNetValidator :

```
@Override
public Boolean caseArc(Arc object) {
    // Contraintes sur Arc
    this.result.recordIfFailed(
        object.getPoids() > 0 ,
        object,
        "Le poids de l'arc doit être supérieur à 0");
    this.result.recordIfFailed(
        (object.getSource() instanceof Place && object.getDestination() instanceof Transition) || (object.getDestination()
        object,
        "l'arc ne peut pas aller d'une place vers une autre place et d'une transition vers une autre transition");

    return null;
}
```

figure Implémentation des contraintes sur Arc : (i) liaison Place↔Transition uniquement ; (ii) poids ≥ 1 .

Chapitre 4

Transformation Modèle à Modèle

4.1 Objectif et principe des transformations

L'objectif de cette chaîne M2M est de passer d'une description *textuelle* du procédé (PDL3) à une représentation *exécutable* et vérifiable (réseau de Pétri), via un modèle intermédiaire structuré (SimplePDL). Nous suivons le pipeline suivant :

$$PDL3 \text{ ATL } SimplePDL \text{ ATL } PetriNet.$$

Les transformations sont implémentées en **ATL** dans l'environnement Eclipse. Elles respectent des règles de correspondance explicites entre classes sources et cibles, puis agrègent les éléments au niveau **Process**.

4.2 Transformation PDL3 → SimplePDL (ATL)

Cette première transformation structure un modèle PDL3 (issu de l'éditeur Xtext) en un modèle **EMF** conforme au métamodèle **SimplePDL**. Les points clés sont :

- **Process** (PDL3) → **Process** (SimplePDL) : même nom, agrégation de tous les éléments convertis ;
- **WorkDefinitionList.wds** → **WorkDefinition** ;
- **WorkSequence** : mapping du type PDL3 (s2s, f2s, s2f, f2f) vers l'énumération SimplePDL (startToStart, finishToStart, startToFinish, finishToFinish) ;
- **Supply** → **Ressource** (nom, quantité) ;
- **Requirement** → **RessourceUsage** (références vers la tâche et la ressource, amount).

```
rule Process3toSimple {
  from s : PDL3Process
  to   sp : SimplePDLProcess {
    name <- s.name
    processElements <-
      s.defs.wds->collect(w | thisModule.resolveTemp(w, 'tw'))
      ->union( s.seqs.wss->collect(s | thisModule.resolveTemp(s, 'tw')) )
      ->union( s.supplies->collect(r | thisModule.resolveTemp(r, 'tres')) )
      ->union( s.req->collect(r | thisModule.resolveTemp(r, 'tre')) )
  }
}

rule WD3toSimple {
  from w : PDL3WorkDefinition
  to   twd : SimplePDLWorkDefinition {
    name <- w.name
  }
}

rule WS3toSimple {
  from s : PDL3WorkSequence
  to   tws : SimplePDLWorkSequence {
    predecessor <- thisModule.resolveTemp(s.predecessor, 'tw')
    successor <- thisModule.resolveTemp(s.successor, 'tw')
    linkType <- thisModule.mapWS(s.type)
  }
}

helper def mapWS(x : PDL3WorkSequenceType) : SimplePDLWorkSequenceType =
  if x = #s2s then #startToStart
  else if x = #f2f then #finishToFinish
  else if x = #s2f then #startToFinish
  else #finishToStart
  endif endif endif
```

Extrait des règles ATL PDL3→SimplePDL

```

1 process : simple
2 workdefinitions :
3   tache1; tache2;
4
5 supplies :
6   Papier(10); Imprimante(1);
7
8 worksequences :
9   tache1 f2s tache2;
10
11 requirements :
12 tache2 needs Papier x 2;
13 tache2 needs Imprimante x 1;
14

```

Exemple d'entrée PDL3 (éditeur Xtext)
Résultat SimplePDL (éditeur EMF) :

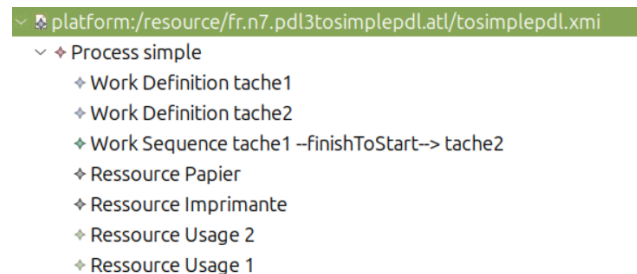


FIGURE 4.1 – Modèle SimplePDL généré (vue arborescente)

4.3 Transformation SimplePDL → PetriNet (ATL)

La seconde transformation produit un **réseau de Pétri** exécutable à partir de **SimplePDL**. Nous avons adopté un schéma standard :

Mapping d'une WorkDefinition

Chaque *WorkDefinition* génère un sous-réseau local composé de :

- 4 **places** : *name_ready*, *name_started*, *name_running*, *name_finished* (avec *ready* initialisé à 1 jeton);
- 2 **transitions** : *name_start*, *name_finish*;

- 5 arcs :
 1. ready $\rightarrow$ start (consommation d’aptitude à démarrer),
 2. start $\rightarrow$ running,
 3. start $\rightarrow$ started (marque “démarrée”),
 4. running $\rightarrow$ finish,
 5. finish $\rightarrow$ finished.

Mapping d’une WorkSequence

Chaque WorkSequence devient un **arc de type Lecture (ReadArc)** entre une place de la tâche prédécesseur et une transition de la tâche successeur, selon le type :

- startToStart : pred.started $\rightarrow$ succ.start
- startToFinish : pred.started $\rightarrow$ succ.finish
- finishToStart : pred.finished $\rightarrow$ succ.start
- finishToFinish : pred.finished $\rightarrow$ succ.finish

Mapping des ressources

- **Ressource $\rightarrow$ Place** : une place par ressource ; le nombre de jetons initial correspond à la quantity de la ressource ;
- **RessourceUsage $\rightarrow$ 2 arcs (poids = amount)** :
 1. Place(ressource) $\rightarrow$ Transition(tâche_start) — consommation au démarrage ;
 2. Transition(tâche_finish) $\rightarrow$ Place(ressource) — restitution à la fin.

Capture de la transformation SimplePDL $\rightarrow$ PetriNet :

```
rule WD2PetriNet {
  from wd : SimplePDLWorkDefinition
  to place_ready : PNPlace(petriNet <- wd.getProcess(), name <- wd.name + 'ready', jetons <- 1),
  place_started : PNPlace(petriNet <- wd.getProcess(), name <- wd.name + 'started'),
  place_running : PNPlace(petriNet <- wd.getProcess(), name <- wd.name + 'running'),
  place_finished : PNPlace(petriNet <- wd.getProcess(), name <- wd.name + 'finished'),
  transition_start : PNTransition(petriNet <- wd.getProcess(), name <- wd.name + 'start'),
  transition_finish : PNTransition(petriNet <- wd.getProcess(), name <- wd.name + 'finish'),
  arc_ready_start : PNReadArc(petriNet <- wd.getProcess(), source <- place_ready, destination <- transition_start),
  arc_start_running : PNReadArc(petriNet <- wd.getProcess(), source <- transition_start, destination <- place_running),
  arc_start_started : PNReadArc(petriNet <- wd.getProcess(), source <- transition_start, destination <- place_started),
  arc_running_finish : PNReadArc(petriNet <- wd.getProcess(), source <- place_running, destination <- transition_finish),
  arc_finish_finished : PNReadArc(petriNet <- wd.getProcess(), source <- transition_finish, destination <- place_finished)
}

rule WS2ReadArc {
  from ws : SimplePDLWorkSequence
  to readarc : PNReadArc(petriNet <- ws.getProcess(), nature <- #Lecture, source <- thisModule.resolveTemp(ws.predecessor, 'place_' +
    (if ws.linkType = #startToStart or ws.linkType = #startToFinish then
      'started'
    else
      'finished'
    endif)), destination <- thisModule.resolveTemp(ws.successor, 'transition_' +
    (if ws.linkType = #startToStart or ws.linkType = #finishToStart then
      'start'
    else
      'finish'
    endif)))
}

rule RS2PetriNet {
  from rs : SimplePDLResource
  to place_rs : PNPlace(petriNet <- rs.getProcess(), name <- rs.name, jetons <- rs.quantity)
}

rule RS2PetriNet {
  from rsu : SimplePDLResourceUsage
  to arc_rsu_ws : PNReadArc(petriNet <- rsu.getProcess(), poids <- rsu.amount, source <- rsu.ressource, destination <- thisModule.resolveTemp(rsu.workdefinition, 'transition_' + 'start'),
  arc_rsu_ws_f : PNReadArc(petriNet <- rsu.getProcess(), poids <- rsu.amount, source <- thisModule.resolveTemp(rsu.workdefinition, 'transition_' + 'finish'), destination <- rsu.ressource)
}
```

FIGURE 4.2 – Extrait des règles ATL SimplePDL $\rightarrow$ PetriNet (WD, WS, Ressource, RessourceUsage)

Chapitre 5

Transformation Modèle à Texte (Acceleo)

5.1 Principe général

Nous utilisons **Acceleo** pour générer des artefacts textuels à partir des modèles EMF : (i) une vue **HTML** des processus *SimplePDL*, (ii) une représentation **Graphviz DOT** des réseaux de Pétri, (iii) un fichier **.net** compatible TINA. Chaque génération s'appuie sur des *templates .mtl* exécutés depuis Eclipse.

5.2 Template toHTML.mtl : génération d'une vue HTML

Le template `toHTML.mtl` produit une page listant les `WorkDefinition` d'un `Process` et leurs dépendances (`linksToPredecessors`).

5.3 Template toDOT.mtl : export Graphviz DOT

Le template `toDOT.mtl` génère une description **DOT** d'un `PetriNet` : `Place` (cercle, label avec `jetons`), `Transition` (boîte), `Arc` (étiquette `poids`, flèche selon `nature`).
Rendu PDF (Graphviz). Le fichier `.dot` est converti via :

```
dot -Tpdf reseau.dot -o reseau.pdf
```

5.4 Template toTina.mtl : génération du fichier .net

Le template `toTina.mtl` parcourt `PetriNet` et produit un fichier **.net** (syntaxe TINA) : déclarations des `pl` (places, `jetons`), des `tr` (transitions, bornes), et des arcs entrants/sortants avec `poids` ou lecture (?).

0.42

```

<head><title>[aProcess.name/]</title></head>
<body>
<h1>Process "[aProcess.name/]"</h1>
<h2>Work definitions</h2>
[let wds : OrderedSet(WorkDefinition) = aProcess.getWDs() ]
  [if (wds->size() > 0)]
    <ul>
      [for (wd : WorkDefinition | wds)]
        <li>[wd.toHTML()/]</li>
      [/for]
    </ul>
  [else]
    <b>None.</b>
  [/if]
[/let]
</body>
[/file]
[/template]

[query public getWDs(p: Process) : OrderedSet(WorkDefinition) =
  p.processElements->select( e | e.ocIsTypeOf(WorkDefinition) )
  ->collect( e | e.ocAsType(WorkDefinition) )
  ->asOrderedSet()
/]

[query public toState(link: WorkSequenceType) : String =
  if link = WorkSequenceType::startToStart or link = WorkSequenceType::startToFinish then
    'started'
  else
    'finished'
  endif
/]

[template public toHTML(wd : WorkDefinition) post (trim()) ]
[wd.name /] [for (ws: WorkSequence | wd.linksToPredecessors)
  before ( ' requires ' ) separator ( ', ' ) after ( '.' )
][ws.predecessor.name /] to be [ws.linkType.toState()
/][/for]
[/template]

```

FIGURE 5.1 – Extrait du template toHTML.mtl

0.42

Process "cheikh"

Work definitions

- a1
- a2 requires a1 to be started.

FIGURE 5.2 – Sortie HTML générée (*Process "cheikh"*)

FIGURE 5.3 – Génération HTML depuis un modèle SimplePDL

```

[query public toState(nature: ArcType) : String =
  if nature = ArcType::Normal then
    'vee'
  else
    'dot'
  endif
/]

[template public toDot(p : Place)]
  [p.name/] [ '[' /]
  shape=circle
  style=filled
  fillcolor="#D5E8D4"
  label="[p.name/] ([p.jetons/])"
  [' ' /]
  [for (arc : Arc | p.sortants)]
    [p.name/] -> [arc.destination.name/] [ '[' /] arrowhead=[arc.nature.toState()] label = [arc.poids/] [ '[' /]
  [/for]
[/template]

[template public toDot(t : Transition)]
  [t.name/] [ '[' /]
  shape=box
  style=filled
  fillcolor="#FFF2CC"
  label="[t.name/]"
  [' ' /]
  [for (arc : Arc | t.sortants)]
    [t.name/] -> [arc.destination.name/] [ '[' /] arrowhead=[arc.nature.toState()] label = [arc.poids/] [ '[' /]
  [/for]
[/template]

```

FIGURE 5.4 – Extrait du template toDOT.mtl

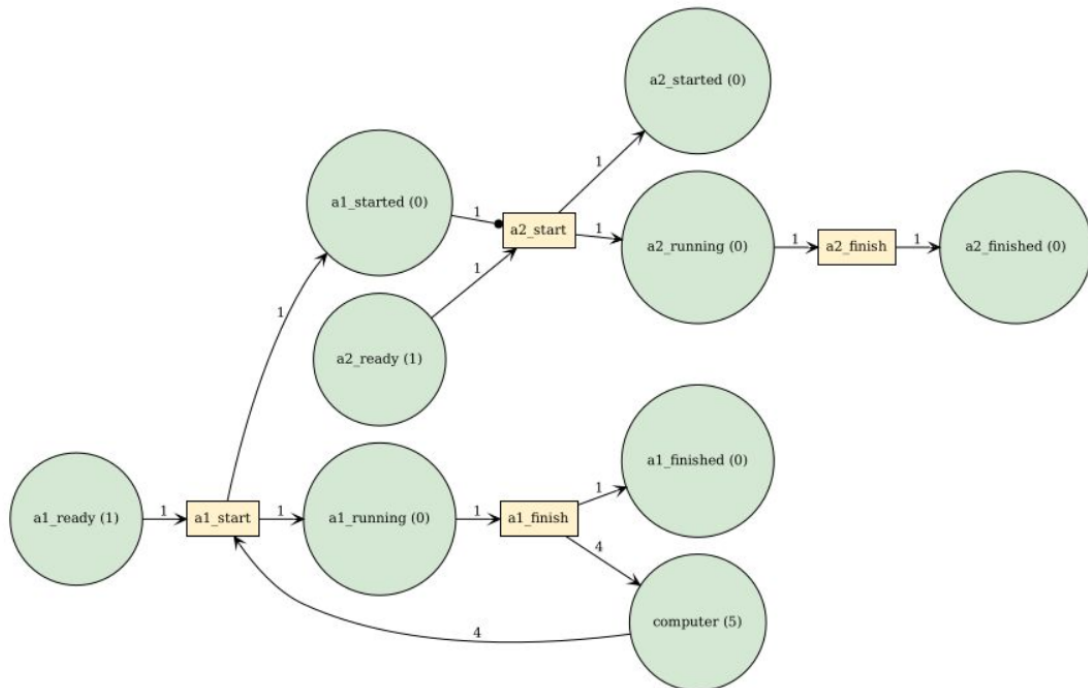


FIGURE 5.5 – Extrait de DOT généré

FIGURE 5.6 – Export DOT du modèle PetriNet

net ifip

```

p1 p1 (1)
p1 p2 (2)
p1 p3 (0)
p1 p4 (0)
p1 p5 (0)
tr t1 [4,9] p1 p2*2 -> p3 p4 p5
tr t2 [0,2] p4 -> p2
tr t3 [1,1] p5 -> p2
tr t4 [0,2] p3 p5?1 -> p3
tr t5 [0,3] p3 -> p1

```

FIGURE 5.7 – Extrait du fichier .net généré

0.42

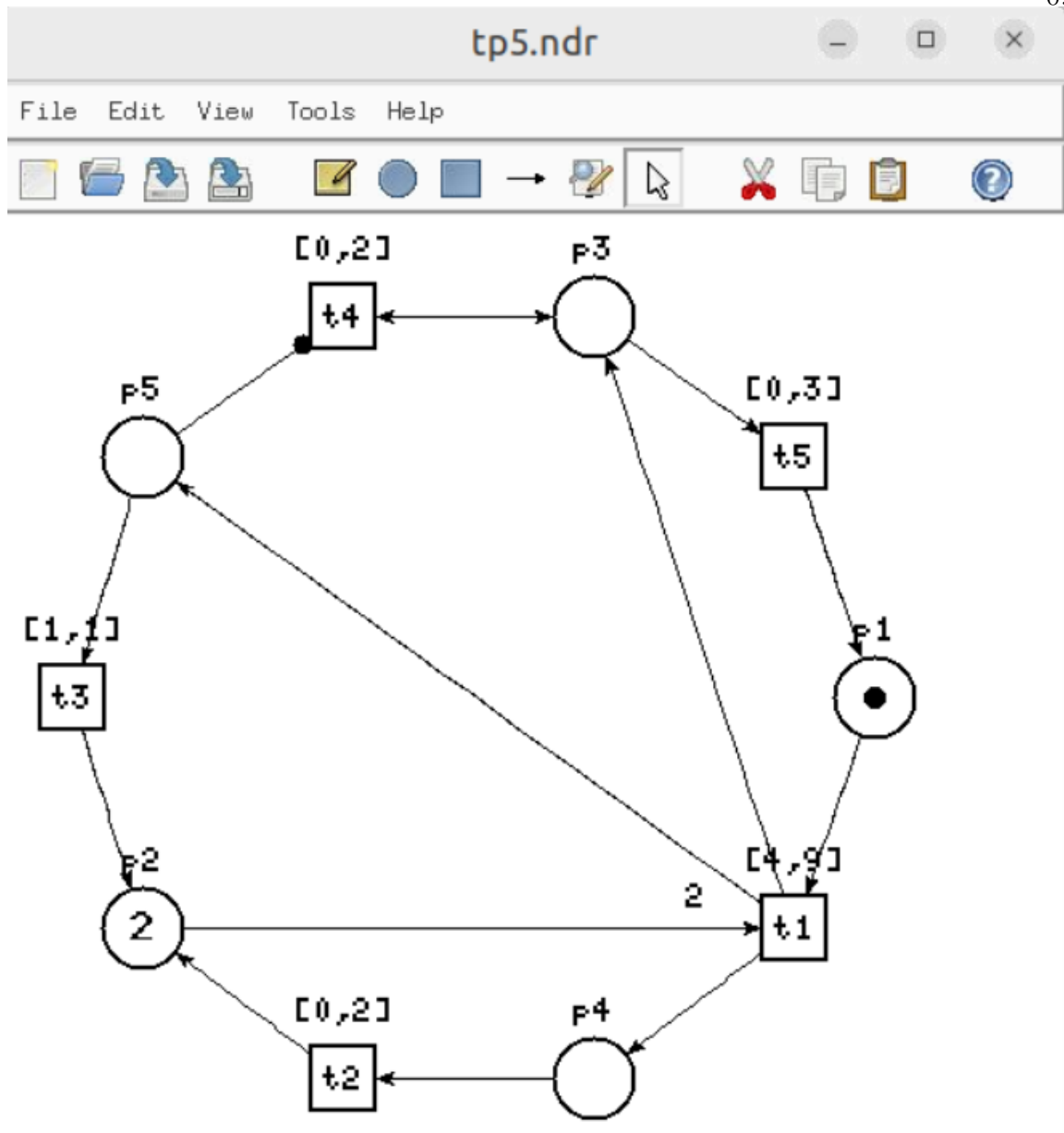


FIGURE 5.8 – Rendu NetDraw du même réseau

FIGURE 5.9 – Chaîne toTina → NetDraw : du .net au schéma

Chapitre 6

Éditeurs et visualisations

6.1 Rappels technologiques

Sirius (éditeur graphique)

À l’instar d’une syntaxe concrète *textuelle*, une syntaxe concrète *graphique* offre un moyen plus agréable et efficace de visualiser et d’éditer un modèle. **Eclipse Sirius**, développé par Obeo et Thales, repose sur les technologies EMF et GMF. Il permet de définir une notation graphique pour un langage décrit en **Ecore** et de générer un éditeur graphique intégré à Eclipse (diagrammes, palette d’outils, synchronisation modèle/diagramme).

Xtext (éditeur textuel)

Xtext permet de définir une syntaxe concrète textuelle pour un DSML à partir d’une grammaire (analysable LL(k) et sans récursion gauche). Il peut engendrer un métamodèle ou s’appuyer sur un métamodèle existant si leur structure est proche. Dans notre projet, il est utilisé pour décrire textuellement les modèles PDL3 avant leur transformation vers SimplePDL.

6.2 Éditeur graphique Sirius

- Nous avons conçu un *Viewpoint* Sirius pour **SimplePDL** permettant :
- la représentation graphique des *WorkDefinition*, *WorkSequence*, *Ressource*, *RessourceUsage* et *Guidance* ;
 - une **palette personnalisée** regroupant des outils comme : *WDCreation*, *WSCreation*, *RsCreation*, *RuCreation*, *RS_Ru_Creation*, *Ru_WD_Creation* ;
 - la **synchronisation automatique** entre le diagramme Sirius et le modèle EMF.
- Grâce à cet éditeur graphique, il est possible de :
- créer des tâches (*WorkDefinitions*) comme *Conception*, *Programmation* ou *Validation* ;
 - relier ces tâches via des dépendances (*WorkSequences*) avec différents types de liens (*startToStart*, *finishToFinish*, etc.) ;
 - définir et relier des ressources à leurs usages dans le processus.

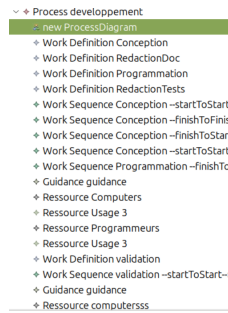


FIGURE 6.1 – Vue arborescente du modèle

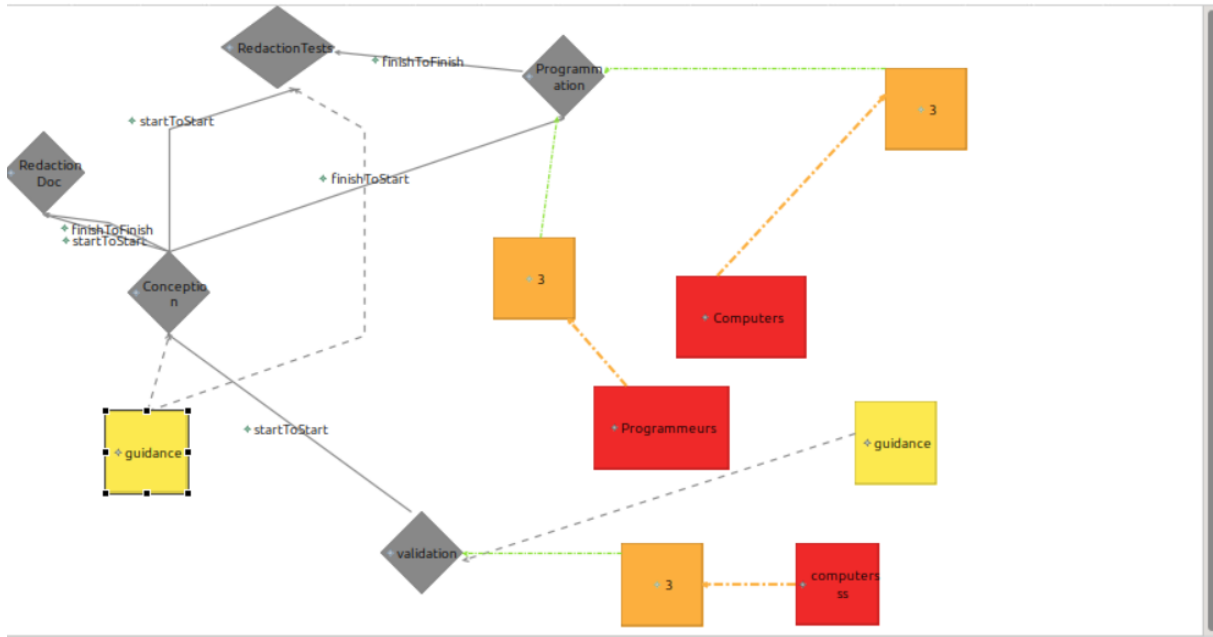


FIGURE 6.2 – Diagramme Sirius du procédé

FIGURE 6.3 – Représentation du procédé dans Sirius (réduite)

Toutes les modifications effectuées sur le diagramme Sirius sont immédiatement visibles dans la structure EMF correspondante, ce qui assure une parfaite cohérence entre les vues.

6.3 Éditeur textuel Xtext

Pour la description textuelle, nous utilisons le langage **PDL3**, défini avec Xtext. La grammaire décrit les éléments du processus :

- **Process** : racine du modèle ;
- **WorkDefinitionList**, **SupplyList**, **WorkSequenceList**, **RequirementList** : listes d'activités, ressources, dépendances et besoins ;
- les références croisées (**[WorkDefinition]**) sont automatiquement résolues.

L'éditeur textuel offre la coloration syntaxique, la complétion automatique et la validation dans Eclipse. Il constitue le point d'entrée textuel de notre chaîne de transformation (PDL3 → SimplePDL).

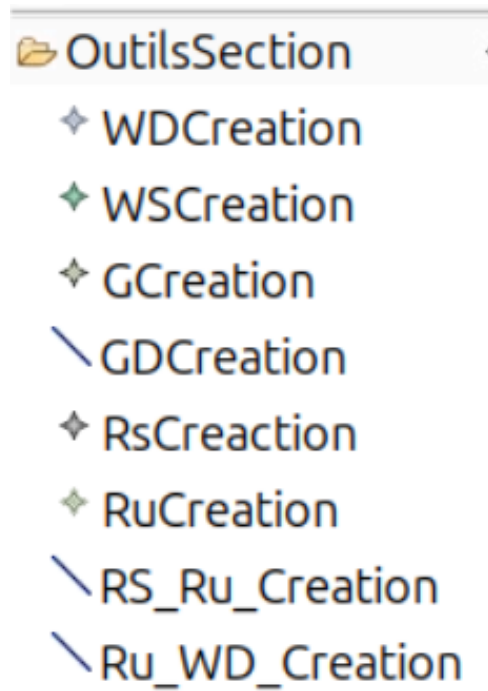


FIGURE 6.4 – Palette d’outils Sirius : création d’activités, ressources et dépendances

6.4 Intégration Eclipse : édition et vérification

Tous nos outils sont intégrés dans le même environnement Eclipse :

1. **Édition** graphique (Sirius) ou textuelle (Xtext) du processus ;
 2. **Validation** du modèle via nos validateurs Java ;
 3. **Transformations ATL** : PDL3 $\rightarrow$ SimplePDL puis SimplePDL $\rightarrow$ PetriNet ;
 4. **Génération** du fichier `.net` avec Acceleo et **vérification LTL** avec TINA/SELT.
- Cette intégration nous permet de modéliser, transformer et vérifier les procédés sans quitter l’environnement Eclipse.

Chapitre 7

Vérification formelle avec TINA / SELT

7.1 Introduction à la logique temporelle linéaire (LTL)

La **logique temporelle linéaire (LTL)** est un langage formel permettant d'exprimer des propriétés sur le comportement d'un système dynamique. Elle permet de vérifier des critères tels que la terminaison, la non-présence de blocages ou encore la précédence entre événements

7.2 Compilation et génération du fichier .ktz

Après la génération du réseau de Pétri (.net) à l'aide du template `toTina.mtl`, nous avons utilisé l'outil **TINA** pour compiler le modèle dans un format exécutable .ktz. La commande utilisée est la suivante :

```
tina exemple.net exemple.ktz
```

Cette étape permet de générer le graphe des marquages et de préparer le modèle pour la vérification formelle.

7.3 Vérification des propriétés LTL

Une fois le fichier .ktz généré, l'outil **SELT** (inclus dans TINA) permet de vérifier les propriétés exprimées en LTL. La commande standard est la suivante :

```
selt -p -S trace.scn exemple.ktz -prelude properities.ltl
```

Cependant, nous n'avons pas pu approfondir cette partie autant que prévu. En effet, la manipulation des fichiers .ltl et la rédaction des formules temporelles nécessitaient une compréhension plus fine de la syntaxe et de la sémantique LTL dans SELT.

Chapitre 8

Partie faite après la soutenance orale

Après la soutenance orale, nous avons poursuivi le travail sur le projet afin de finaliser certaines étapes de la chaîne de transformation. Nous avons principalement réalisé la **transformation ATL de PDL3 vers SimplePDL**, ainsi qu'un début de travail sur les **propriétés LTL**.

Le choix du langage **PDL3** s'est imposé suite aux difficultés rencontrées avec **PDL1** dans Eclipse, qui ne permettait pas d'exécuter correctement les exemples minimaux utilisés pendant les TP. Le métamodèle **PDL3** étant plus complet (intégrant les ressources et leurs usages), il nous a permis de corriger ces problèmes et d'obtenir une transformation fonctionnelle entre **PDL3** et **SimplePDL**.

Cette partie, bien qu'effectuée après la présentation, nous a permis de consolider le projet et de vérifier la cohérence de la chaîne jusqu'à la génération et la vérification des modèles dans TINA.

Conclusion générale

Ce mini-projet d'**Ingénierie Dirigée par les Modèles (IDM)** nous a permis de mettre en pratique l'ensemble des notions abordées durant le module : la conception de métamodèles, les transformations de modèles, la génération de code et la vérification formelle.

Nous avons ainsi construit une **chaîne complète de modélisation et de vérification** d'un processus, allant d'une description textuelle (PDL3) jusqu'à l'analyse de son comportement sous forme de réseau de Pétri (**PetriNet**) et de fichiers exécutables pour l'outil **TINA**. Chaque étape a fait intervenir un outil de la plateforme Eclipse : **Xtext** pour la syntaxe textuelle, **Sirius** pour la visualisation graphique, **ATL** pour les transformations M2M et **Acceleo** pour la génération de code texte.

Le passage à PDL3, réalisé après la soutenance, a représenté un travail conséquent. Il a permis d'intégrer la gestion des ressources et d'obtenir une transformation **PDL3** $\rightarrow$

SimplePDL pleinement fonctionnelle, garantissant une meilleure cohérence entre modèles. Cette extension a consolidé la chaîne de vérification et démontré la faisabilité d'un projet IDM complet.

Nous avons également pu générer les fichiers **.net** compatibles avec **TINA** et effectuer des vérifications formelles simples à l'aide de la logique temporelle linéaire (LTL). Même si cette dernière partie n'a pas pu être explorée en profondeur par manque de temps, nous avons validé le bon fonctionnement global de la chaîne jusqu'à la génération des modèles vérifiables.

Au-delà de l'aspect technique, ce projet nous a permis de développer des compétences précieuses : travail collaboratif, raisonnement modulaire et compréhension globale des flux de transformation dans une approche dirigée par les modèles. Il nous a également sensibilisés à l'importance de la cohérence entre les différents niveaux d'abstraction, de la modélisation jusqu'à la vérification formelle.

En conclusion, ce mini-projet a été une expérience enrichissante, tant sur le plan technique que méthodologique, et nous a offert une vision concrète du potentiel et des défis de l'ingénierie dirigée par les modèles.